

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Curso:

Base de Datos II

Laboratorio 03: External Algorithms

Alumno:

Llerena Silva, Nicolás Alejandro (202110190)

Profesor:

Lovon, Percy

Lima, agosto de 2026

Introducción

Este laboratorio tiene dos mitades. En la primera reviso qué algoritmos de memoria externa usa PostgreSQL cuando lo obligo a trabajar con 64 kB de `work_mem` sobre la base `employees` del Laboratorio 01. En la segunda implemento en Python, sobre archivos binarios paginados, el Two-Phase Multiway Merge Sort y el External Hashing para un `GROUP BY`, y contrasto los resultados contra los del motor.

Las pruebas se corrieron sobre PostgreSQL 14.18 en macOS. Las tablas involucradas son `employee`, con 300 024 filas, y `department_employee`, con 331 603. En la implementación en Python el tamaño de página es de 4096 bytes. Las salidas que aparecen a lo largo del informe son las que devolvieron `psql` y los programas; los archivos completos están en la carpeta evidencia del entregable.

Parte 1: planes de ejecución en PostgreSQL

Cada consulta se ejecutó dos veces, primero con `work_mem` en 64 kB y después en 2 MB, dentro de una transacción y con `EXPLAIN (ANALYZE, BUFFERS)`. El script `capturar_planes.sh` automatiza la captura y deja cada plan en un archivo aparte. Como `EXPLAIN` no informa cuántas pasadas de merge hizo un ordenamiento, para la primera consulta también activé `trace_sort` junto con `client_min_messages = LOG`, que es lo que hace que esa traza interna llegue al cliente, y apagué el paralelismo para poder seguir un solo ordenamiento.

Consulta 1: ORDER BY sobre hire_date

```
psql - employees

$ psql -U postgres -d employees
psql (14.18 (Homebrew))
employees=# BEGIN;
BEGIN
employees=# SET LOCAL work_mem = '64kB';
SET
employees=# EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM employees.employee ORDER BY hire_date;
               QUERY PLAN
Gather Merge  (cost=35105.31..55401.08 rows=176485 width=35) (actual time=82.639..109.241 rows=300024 loops=1)
  Workers Planned: 1
  Workers Launched: 1
  Buffers: shared hit=2519, temp read=6937 written=7519
    -> Sort  (cost=34105.30..34546.51 rows=176485 width=35) (actual time=78.487..86.491 rows=150012 loops=2)
        Sort Key: hire_date
        Sort Method: external merge  Disk: 7192kB
        Buffers: shared hit=2519, temp read=6937 written=7519
        Worker 0: Sort Method: external merge  Disk: 6632kB
        -> Parallel Seq Scan on employee  (cost=0.00..4245.85 rows=176485 width=35) (actual time=0.005..9.191 rows=150012
        loops=2)
            Buffers: shared hit=2481
Planning:
  Buffers: shared hit=87
Planning Time: 0.473 ms
Execution Time: 115.718 ms
(15 rows)

employees=# COMMIT;
COMMIT
employees=# \q
$
```

Plan de la consulta 1 con work_mem = 64kB.

El ordenamiento no cabe en memoria. El nodo Sort reporta el método external merge y vuelca 7192 kB a disco, y el worker que lo acompaña otros 6632 kB; entre ambos el motor leyó 6937 bloques temporales y escribió 7519. Al subir work_mem a 2 MB el plan mantiene exactamente la misma forma y el método sigue siendo externo, porque las 300 024 filas ocupan alrededor de 7 MB una vez proyectadas. Lo que cambia es el trabajo temporal, que baja a 2288 bloques leídos y 2302 escritos, y el tiempo, que pasa de 115,7 ms a 55,0 ms. Para ver el caso en memoria hay que llegar a 64 MB: recién ahí el método cambia a quicksort con 35 729 kB y desaparecen los archivos temporales.

```
psql - employees

$ psql -U postgres -d employees
psql (14.18 (Homebrew))
employees=# BEGIN;
BEGIN
employees=# SET LOCAL work_mem = '2MB';
SET
employees=# EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM employees.employee ORDER BY hire_date;
QUERY PLAN
Gather Merge  (cost=25452.31..45748.08 rows=176485 width=35) (actual time=24.806..49.148 rows=300024 loops=1)
  Workers Planned: 1
  Workers Launched: 1
  Buffers: shared hit=2519, temp read=2288 written=2302
  -> Sort  (cost=24452.30..24893.51 rows=176485 width=35) (actual time=23.583..30.905 rows=150012 loops=2)
    Sort Key: hire_date
    Sort Method: external merge  Disk: 7152kB
    Buffers: shared hit=2519, temp read=2288 written=2302
    Worker 0:  Sort Method: external merge  Disk: 6632kB
    -> Parallel Seq Scan on employee  (cost=0.00..4245.85 rows=176485 width=35) (actual time=0.002..3.792 rows=150012 loops=2)
      Buffers: shared hit=2481
Planning:
  Buffers: shared hit=87
Planning Time: 0.144 ms
Execution Time: 54.966 ms
(15 rows)

employees=# COMMIT;
COMMIT
employees=# \q
$
```

El mismo plan con work_mem = 2MB.

```
psql - employees

$ psql -U postgres -d employees
psql (14.18 (Homebrew))
employees=# BEGIN;
BEGIN
employees=# SET LOCAL work_mem = '64MB';
SET
employees=# EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM employees.employee ORDER BY hire_date;
QUERY PLAN
Sort  (cost=32775.50..33525.56 rows=300024 width=35) (actual time=31.464..42.485 rows=300024 loops=1)
  Sort Key: hire_date
  Sort Method: quicksort  Memory: 35729kB
  Buffers: shared hit=2484
  -> Seq Scan on employee  (cost=0.00..5481.24 rows=300024 width=35) (actual time=0.002..7.259 rows=300024 loops=1)
    Buffers: shared hit=2481
Planning:
  Buffers: shared hit=87
Planning Time: 0.124 ms
Execution Time: 46.906 ms
(10 rows)

employees=# COMMIT;
COMMIT
employees=# \q
$
```

Con 64 MB el ordenamiento ya cabe en memoria.

```
bash - labs/3

-- work_mem = 64kB
$ grep -c 'finished writing run' evidencia/q1_order_by_64kB_trace_sort.txt
586
$ grep -c 'merge step' evidencia/q1_order_by_64kB_trace_sort.txt
118
$ grep -E 'external sort with|read buffers' evidencia/q1_order_by_64kB_trace_sort.txt
LOG:  worker -1 switching to external sort with 7 tapes: CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s
LOG:  worker -1 using 56 KB of memory for read buffers among 6 input tapes

-- work_mem = 2MB
$ grep -c 'finished writing run' evidencia/q1_order_by_2MB_trace_sort.txt
16
$ grep -c 'merge step' evidencia/q1_order_by_2MB_trace_sort.txt
2
$ grep -E 'external sort with|read buffers' evidencia/q1_order_by_2MB_trace_sort.txt
LOG:  worker -1 switching to external sort with 8 tapes: CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s
LOG:  worker -1 using 1975 KB of memory for read buffers among 7 input tapes

$
```

Conteo de runs y merge steps sobre la traza de trace_sort.

¿Qué algoritmo de memoria externa aparece cuando la RAM es insuficiente? El external merge sort, que es la versión multipasada del Two-Phase Multiway Merge Sort. PostgreSQL llena work_mem, ordena ese bloque con quicksort y lo escribe como un run en una cinta temporal, que es la fase 1; después mezcla las cintas con un árbol de torneo, que cumple el papel del min-heap de la fase 2.

¿Cuántos merge passes realizó? La traza con 64 kB muestra 586 runs y 118 merge steps repartidos sobre 7 cintas, seis de entrada y una de salida. Eso equivale a cuatro pasadas: 586 entre 6 dan unas 98 mezclas en el primer nivel, después 17, luego 3 y una final, que suman 119 y coinciden con las 118 observadas. Con 2 MB los runs bajan a 16 y alcanzan 2 merge steps. La relación con la fase 1 es directa: si hay $F = B - 1$ cintas de entrada, el número de pasadas es el logaritmo en base F de la cantidad de runs, y solo cuando esos runs caben en las cintas disponibles el algoritmo es de dos fases en el sentido estricto. Más memoria produce runs más largos, menos runs y por lo tanto menos pasadas.

¿Cómo cambia el costo al aumentar work_mem? El costo estimado del plan baja de 55 401 a 45 748 entre 64 kB y 2 MB, y el tiempo real se reduce a la mitad, aunque el método siga siendo externo: lo que mejora es el número de pasadas, y eso se ve en el I/O temporal, que cae más de tres veces. Con 64 MB el costo total llega a 33 526, el tiempo a 46,9 ms y el planificador incluso

abandona el plan paralelo, porque ya no necesita repartir el ordenamiento entre dos procesos.

Consulta 2: GROUP BY sobre from_date

```
psql - employees

$ psql -U postgres -d employees
psql (14.18 (Homebrew))
employees=# BEGIN;
BEGIN
employees=# SET LOCAL work_mem = '64kB';
SET
employees=# EXPLAIN (ANALYZE, BUFFERS) SELECT from_date, COUNT(*) FROM employees.department_employee GROUP BY from_date;
QUERY PLAN
Finalize GroupAggregate (cost=29628.35..30418.62 rows=6079 width=12) (actual time=40.347..43.023 rows=6393 loops=1)
  Group Key: from_date
  Buffers: shared hit=2121, temp read=1848 written=2962
  -> Gather Merge (cost=29628.35..30327.43 rows=6079 width=12) (actual time=40.344..42.396 rows=12767 loops=1)
    Workers Planned: 1
    Workers Launched: 1
    Buffers: shared hit=2121, temp read=1848 written=2962
    -> Sort (cost=28628.34..28643.53 rows=6079 width=12) (actual time=39.409..39.688 rows=6384 loops=2)
      Sort Key: from_date
      Sort Method: external merge  Disk: 184kB
      Buffers: shared hit=2121, temp read=1848 written=2962
      Worker 0: Sort Method: external merge  Disk: 184kB
      -> Partial HashAggregate (cost=25032.66..28141.28 rows=6079 width=12) (actual time=19.517..38.634 rows=6384
        loops=2)
        Group Key: from_date
        Planned Partitions: 4  Batches: 21  Memory Usage: 93kB  Disk Usage: 6920kB
        Buffers: shared hit=2113, temp read=1784 written=2884
        Worker 0: Batches: 33  Memory Usage: 93kB  Disk Usage: 4056kB
        -> Parallel Seq Scan on department_employee (cost=0.00..4063.61 rows=195061 width=4) (actual
          time=0.002..4.218 rows=165802 loops=2)
          Buffers: shared hit=2113
    Planning:
      Buffers: shared hit=131
    Planning Time: 0.259 ms
    Execution Time: 44.073 ms
    (23 rows)

employees=# COMMIT;
COMMIT
employees=# \q
$
```

Plan de la consulta 2 con work_mem = 64kB.

Este plan es interesante porque con poca memoria aparecen los dos algoritmos a la vez. La agregación parcial de cada proceso se resuelve con un Partial HashAggregate que desborda a disco, y los resultados parciales se combinan con Sort más Finalize GroupAggregate, porque Gather Merge exige entrada ordenada. Con 2 MB los 6393 grupos entran en 721 kB, el nodo pasa a Batches: 1 y el Sort desaparece del plan.

```
psql - employees

$ psql -U postgres -d employees
psql (14.18 (Homebrew))
employees=# BEGIN;
BEGIN
employees=# SET LOCAL work_mem = '2MB';
SET
employees=# EXPLAIN (ANALYZE, BUFFERS) SELECT from_date, COUNT(*) FROM employees.department_employee GROUP BY from_date;
QUERY PLAN
Finalize HashAggregate (cost=6738.00..6798.79 rows=6079 width=12) (actual time=19.063..19.299 rows=6393 loops=1)
  Group Key: from_date
  Batches: 1 Memory Usage: 721kB
  Buffers: shared hit=2113
  -> Gather (cost=6038.91..6707.60 rows=6079 width=12) (actual time=17.095..18.163 rows=12766 loops=1)
    Workers Planned: 1
    Workers Launched: 1
    Buffers: shared hit=2113
    -> Partial HashAggregate (cost=5038.91..5099.70 rows=6079 width=12) (actual time=16.193..16.416 rows=6383
        loops=2)
      Group Key: from_date
      Batches: 1 Memory Usage: 721kB
      Buffers: shared hit=2113
      Worker 0: Batches: 1 Memory Usage: 721kB
      -> Parallel Seq Scan on department_employee (cost=0.00..4063.61 rows=195061 width=4) (actual
        time=0.002..4.486 rows=165802 loops=2)
        Buffers: shared hit=2113
Planning:
  Buffers: shared hit=131
Planning Time: 0.228 ms
Execution Time: 19.479 ms
(19 rows)

employees=# COMMIT;
COMMIT
employees=# \q
$
```

El mismo plan con work_mem = 2MB.

¿Hash Aggregate o Sort + Group Aggregate? Las dos. Desde la versión 13 PostgreSQL ya no descarta el hash aggregate cuando no cabe en memoria, sino que lo particiona: el plan muestra Planned Partitions: 4, Batches: 21, 93 kB de memoria y 6920 kB en disco. El Sort que aparece encima no es una alternativa al hash, sino el paso que ordena los resultados parciales antes de combinarlos.

Cuando aparece Sort + Group Aggregate, internamente se está usando el mismo external merge sort de la consulta anterior; aquí el nodo reporta external merge con 184 kB en disco. Es la agregación basada en ordenamiento: se ordena por la clave de grupo y una sola pasada secuencial acumula los conteos de cada corrida de claves iguales.

Cuando aparece Hash Aggregate con batches mayores que uno, lo que está ocurriendo es external hashing, en su variante grace o híbrida. Es exactamente lo que implementé en `external_hashing.py`: la fase 1 reparte las tuplas en particiones según una función hash sobre `from_date` y las escribe en archivos temporales, y la fase 2 procesa una partición a la vez construyendo la tabla

hash en RAM. El número de batches es el número de particiones, y crece cuando la tabla hash no cabe en work_mem.

Consulta 3: JOIN sobre employee_id

```
psql - employees

$ psql -U postgres -d employees
psql (14.18 (Homebrew))
employees=# BEGIN;
BEGIN
employees=# SET LOCAL work_mem = '64kB';
SET
employees=# EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM employees.employee e JOIN employees.department_employee d ON
d.employee_id = e.id;
                                QUERY PLAN
Hash Join  (cost=11575.54..24105.04 rows=331603 width=56) (actual time=192.561..532.091 rows=331603 loops=1)
  Hash Cond: (d.employee_id = e.id)
  Buffers: shared hit=4594, temp read=4422 written=4422
  -> Seq Scan on department_employee d  (cost=0.00..5429.03 rows=331603 width=21) (actual time=0.002..7.794 rows=331603
    loops=1)
    Buffers: shared hit=2113
  -> Hash  (cost=5481.24..5481.24 rows=300024 width=35) (actual time=164.079..164.080 rows=300024 loops=1)
    Buckets: 1024  Batches: 512  Memory Usage: 50kB
    Buffers: shared hit=2481, temp written=1867
    -> Seq Scan on employee e  (cost=0.00..5481.24 rows=300024 width=35) (actual time=0.002..8.103 rows=300024
      loops=1)
      Buffers: shared hit=2481
Planning:
  Buffers: shared hit=302
Planning Time: 0.422 ms
Execution Time: 537.791 ms
(14 rows)

employees=# COMMIT;
COMMIT
employees=# \q
$
```

Plan de la consulta 3 con work_mem = 64kB.

El algoritmo elegido con RAM limitada es un Hash Join en su variante externa, el *external hash join*. La tabla de build es *employee* y no entra en 64 kB, así que el nodo Hash la parte en 512 batches escritos a disco y la relación de probe se reparte con la misma función hash, lo que suma 4422 bloques temporales leídos y otros tantos escritos. Con 2 MB el mismo plan usa 16 batches y 1526 kB de memoria, y el tiempo cae de 537,8 ms a 110,8 ms.

```
psql - employees

$ psql -U postgres -d employees
psql (14.18 (Homebrew))
employees=# BEGIN;
BEGIN
employees=# SET LOCAL work_mem = '2MB';
SET
employees=# EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM employees.employee e JOIN employees.department_employee d ON
    d.employee_id = e.id;

               QUERY PLAN
Hash Join  (cost=11575.54..24105.04 rows=331603 width=56) (actual time=26.985..105.779 rows=331603 loops=1)
  Hash Cond: (d.employee_id = e.id)
  Buffers: shared hit=4594, temp read=3636 written=3636
  -> Seq Scan on department_employee d  (cost=0.00..5429.03 rows=331603 width=21) (actual time=0.003..8.069 rows=331603
    loops=1)
    Buffers: shared hit=2113
  -> Hash  (cost=5481.24..5481.24 rows=300024 width=35) (actual time=26.635..26.636 rows=300024 loops=1)
    Buckets: 32768  Batches: 16  Memory Usage: 1526kB
    Buffers: shared hit=2481, temp written=1945
    -> Seq Scan on employee e  (cost=0.00..5481.24 rows=300024 width=35) (actual time=0.002..7.880 rows=300024
      loops=1)
      Buffers: shared hit=2481
Planning:
  Buffers: shared hit=302
Planning Time: 0.398 ms
Execution Time: 110.816 ms
(14 rows)

employees=# COMMIT;
COMMIT
employees=# \q
$
```

El mismo plan con work_mem = 2MB.

Esos batches son particiones, y se corresponden uno a uno con la fase 1 del external hashing: PostgreSQL elige la potencia de dos más pequeña tal que un batch de la relación de build quepa en work_mem, y como reparte ambas relaciones con la misma función, las tuplas que hacen match caen siempre en el mismo batch. Eso permite que la fase 2 procese cada par de particiones de forma independiente en memoria. Pasar de 16 a 512 particiones multiplicó el tiempo casi por cinco por el costo de escribir y releer tantos archivos temporales.

Merge Join no apareció en ninguno de los dos escenarios. Si el planificador lo hubiera elegido, habría necesitado dejar ambas entradas ordenadas por employee_id antes del join, sea con dos nodos Sort, con el gasto de disco que eso implica con 64 kB, o recorriendo los índices de clave primaria, que ya entregan las filas en ese orden.

Parte 2: implementación en Python

Estructura del heap file

Los dos archivos binarios se generan desde PostgreSQL con export_data.py, que exporta cada tabla a CSV y la vuelca al formato paginado. La página 0

guarda la cabecera del archivo: número mágico, tamaño de página, tamaño de registro, cantidad de páginas y de registros, la cadena de formato de struct y los nombres de los campos. Guardar el formato dentro del archivo es lo que permite que `read_page(heap_path, page_id, page_size)` devuelva tuplas ya tipadas sin recibir el formato como parámetro, tal como pide la firma del enunciado.

Cada página de datos empieza con cuatro bytes que indican cuántos registros contiene y sigue con registros de longitud fija que nunca cruzan el límite de la página. Para `employee` el registro ocupa 59 bytes y entran 69 por página, de modo que la tabla ocupa 4349 páginas; para `department_employee` son 32 bytes, 127 registros por página y 2612 páginas. Las cuatro funciones que pide el enunciado son la interfaz pública, pero por dentro se apoyan en una clase `HeapFile` que mantiene el descriptor abierto y lleva la cuenta de páginas leídas y escritas. Ese contador es el que alimenta las métricas de I/O, y como los runs y las particiones temporales también son heap files, todo el I/O queda medido con el mismo instrumento.

```
bash - labs/3

$ python3 heap_file.py
employee
  heap_path      : data/employee.bin
  page_size      : 4096
  record_format  : <q10s14s16s1s10s
  record_size    : 59
  fields         : ['id', 'birth_date', 'first_name', 'last_name', 'gender', 'hire_date']
  records_per_page: 69
  records        : 300024
  pages          : 4349
  bytes          : 17817600
  primer registro : (10001, '1953-09-02', 'Georgi', 'Facello', 'M', '1986-06-26')
  ultimo registro : (499999, '1958-05-01', 'Sachin', 'Tsukuda', 'M', '1997-11-30')
  paginas        : 4349
department_employee
  heap_path      : data/department_employee.bin
  page_size      : 4096
  record_format  : <q4s10s10s
  record_size    : 32
  fields         : ['employee_id', 'department_id', 'from_date', 'to_date']
  records_per_page: 127
  records        : 331603
  pages          : 2612
  bytes          : 10702848
  primer registro : (10001, 'd005', '1986-06-26', '9999-01-01')
  ultimo registro : (499999, 'd004', '1997-11-30', '9999-01-01')
  paginas        : 2612
$
```

Salida de heap_file.py sobre los dos archivos exportados.

External sorting: Two-Phase Multiway Merge Sort

La fase 1, en `generate_runs`, lee exactamente $B = \text{BUFFER_SIZE} // \text{PAGE_SIZE}$ páginas, las ordena en memoria por la clave y las escribe como un run. El número de runs es el techo de P entre B , y el programa lo verifica con un `assert`. La fase 2, en `multiway_merge`, mezcla los runs con un min-heap de `heapq` usando $B - 1$ buffers de entrada y uno de salida: cada lector mantiene una sola página en memoria y pide la siguiente cuando la agota, y el buffer de salida se vacía a disco apenas se llena, así que en ningún momento hay más de B páginas residentes.

Con este volumen de datos el número de runs supera a los buffers de entrada disponibles en las configuraciones chicas: con 64 KB salen 272 runs contra 15 buffers, así que una sola pasada de mezcla es imposible y la fase 2 se generaliza a varias pasadas, igual que hace PostgreSQL con sus cintas. Desde 512 KB el algoritmo sí es estrictamente de dos fases, porque los 34 runs caben en los 127 buffers, y ahí el I/O toca su mínimo teórico de $4P$ páginas. La salida se valida recorriéndola entera: orden no decreciente y los 300 024 registros completos.

```
bash - labs/3

$ python3 external_sort.py
TPMMS sobre data/employee.bin ordenando por hire_date
buffer_size      : 65536
buffer_pages     : 16
fan_in           : 15
input_pages      : 4349
runs_generated   : 272
merge_passes     : 3
pages_read       : 17396
pages_written    : 17396
time_phase1_sec  : 0.5378
time_phase2_sec  : 1.6641
time_total_sec   : 2.2019
runs esperados   : ceil(4349/16) = 272
salida ordenada  : data/employee_sorted.bin (300024 registros verificados)
primer registro  : (110022, '1956-09-12', 'Margareta', 'Markovitch', 'M', '1985-01-01')
ultimo registro  : (463807, '1964-06-12', 'Bikash', 'Covnot', 'M', '2000-01-28')
$
```

Salida de external_sort.py con BUFFER_SIZE = 64 KB.

External hashing para el GROUP BY

La fase 1, en `partition_data`, recorre el heap file página a página con un solo buffer de entrada y reparte cada tupla en $k = B - 1$ particiones según $h_p(\text{from_date})$ módulo k , con un buffer de salida de una página por partición. La fase 2, en `aggregate_partitions`, procesa una partición a la vez y construye una tabla hash residente con una función distinta, h_r , implementada con encadenamiento y duplicación de buckets cuando el factor de carga pasa de dos. Las dos funciones son `crc32` con semillas diferentes, lo que las hace deterministas entre ejecuciones; `hash()` de Python no sirve para esto porque está aleatorizado por proceso.

Como las particiones son disjuntas por clave, cada valor de `from_date` aparece en una sola partición y los conteos se unen sin combinar parciales. Vale la pena notar que la memoria residente no depende del tamaño de la partición sino de sus grupos distintos: con 64 KB la tabla más grande tuvo 458 entradas, unos 7 KB entre claves y contadores, muy por debajo del presupuesto, así que no hizo falta particionar de forma recursiva. El resultado se compara contra el archivo `group_by_referencia.csv`, que es la misma consulta ejecutada en PostgreSQL, y la comparación es un `assert` dentro del programa: los 6393 grupos y los 331 603 conteos coinciden exactamente.

```
bash - labs/3

$ python3 external_hashing.py
External hashing GROUP BY from_date sobre data/department_employee.bin
  buffer_size      : 65536
  buffer_pages     : 16
  partitions_created : 15
  groups           : 6393
  max_groups_in_partition : 458
  partition_pages_min : 157
  partition_pages_max : 190
  pages_read       : 5229
  pages_written     : 2617
  time_phase1_sec   : 0.4272
  time_phase2_sec   : 0.2716
  time_total_sec    : 0.6988
  suma de conteos   : 331603
  1985-01-01 -> 9
  1985-01-14 -> 1
  1985-02-01 -> 8
  1985-02-02 -> 54
  1985-02-03 -> 61
  verificacion      : identico a PostgreSQL (6393 grupos)
$
```

Salida de `external_hashing.py` con `BUFFER_SIZE = 64 KB`. La última línea es la verificación contra PostgreSQL.

Análisis de rendimiento

Cada configuración se ejecutó con `benchmark.py`, que además vuelve a validar el orden de la salida y la igualdad del GROUP BY. Las tres primeras filas son las que pide el enunciado; agregué 512 KB y 1 MB para mostrar el punto en que el sort se vuelve estrictamente de dos fases.

```
bash - labs/3

$ python3 benchmark.py
 64 KB  B= 16  runs= 272  passes=3  sort= 2.21s  io= 34792 | particiones= 15  hash= 0.68s  io= 7846
128 KB  B= 32  runs= 136  passes=2  sort= 1.67s  io= 26094 | particiones= 31  hash= 0.70s  io= 7864
256 KB  B= 64  runs= 68   passes=2  sort= 1.67s  io= 26094 | particiones= 63  hash= 0.71s  io= 7896
512 KB  B= 128 runs= 34   passes=1  sort= 1.15s  io= 17396 | particiones= 127 hash= 0.74s  io= 7968
1024 KB B= 256 runs= 17   passes=1  sort= 1.13s  io= 17396 | particiones= 255 hash= 0.78s  io= 8090

resultados en resultados/
$
```

Salida de `benchmark.py` con las cinco configuraciones.

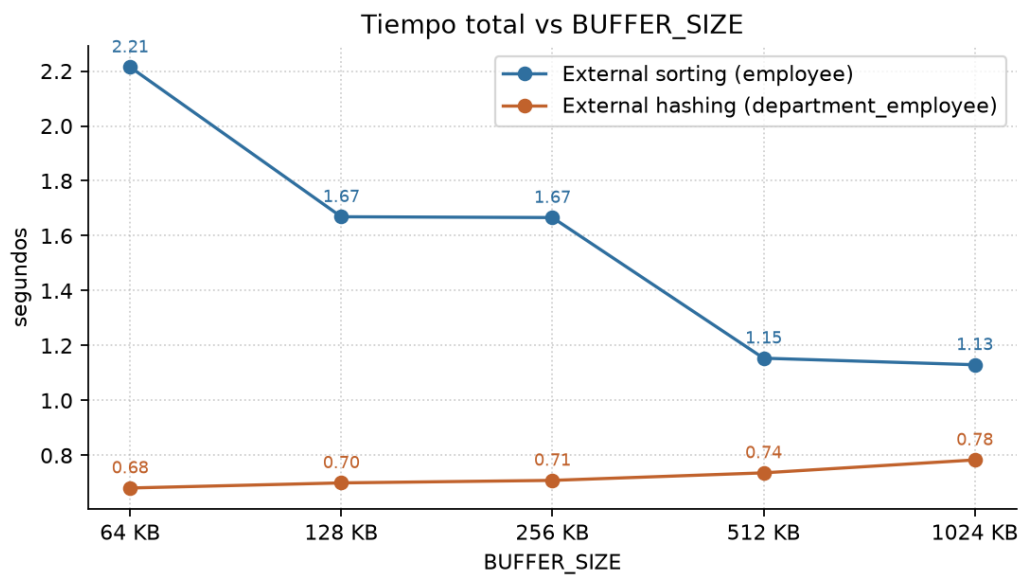
BUFFER_SIZE	B	Runs	Pasadas	Fase 1 (s)	Fase 2 (s)	Total (s)	I/O (págs.)
64 KB	16	272	3	0,56	1,66	2,21	34 792

128 KB	32	136	2	0,55	1,12	1,67	26 094
256 KB	64	68	2	0,57	1,10	1,67	26 094
512 KB	128	34	1	0,56	0,60	1,15	17 396
1024 KB	256	17	1	0,57	0,56	1,13	17 396

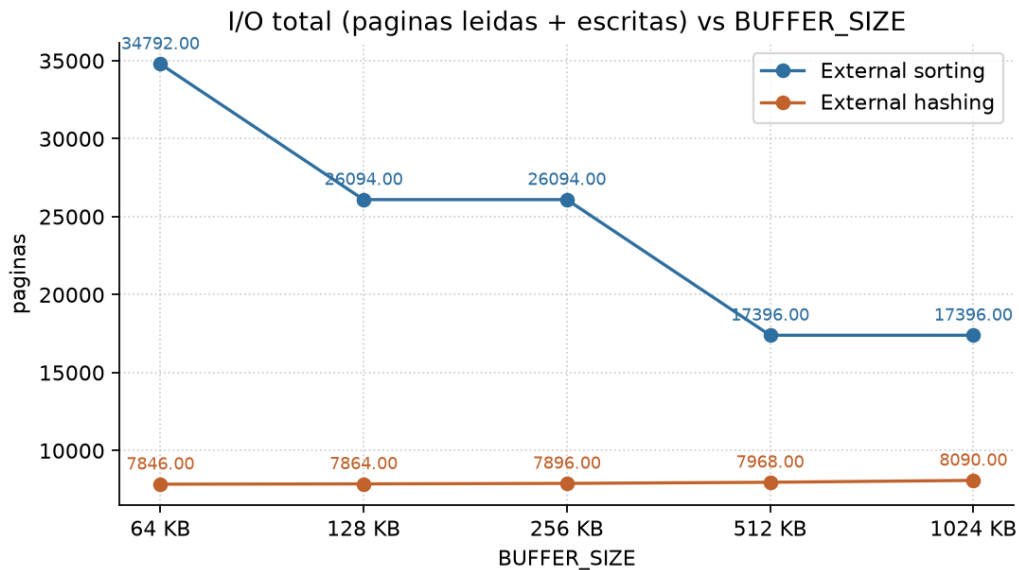
External sorting sobre employee, ordenando por hire_date (P = 4349 páginas).

BUFFER_SIZE	B	Particiones	Grupos máx.	Fase 1 (s)	Fase 2 (s)	Total (s)	I/O (págs.)
64 KB	16	15	458	0,42	0,27	0,68	7 846
128 KB	32	31	251	0,43	0,27	0,70	7 864
256 KB	64	63	126	0,43	0,28	0,71	7 896
512 KB	128	127	74	0,45	0,28	0,74	7 968
1024 KB	256	255	41	0,49	0,29	0,78	8 090

External hashing sobre department_employee, agrupando por from_date (P = 2612 páginas).



Tiempo total contra BUFFER_SIZE para los dos algoritmos.



Páginas leídas más escritas contra BUFFER_SIZE.

El número de runs de la fase 1 se reduce a la mitad cada vez que se duplica BUFFER_SIZE, porque es el techo de P entre B y B crece de forma proporcional: 272, 136, 68, 34 y 17. El tiempo, en cambio, no se reduce a la mitad. La fase 1 siempre lee y escribe las mismas $2P$ páginas y su costo se mantiene alrededor de medio segundo; lo que mejora es la fase 2, cuyas pasadas caen de tres a dos y después a una. Por eso 128 KB y 256 KB rinden casi igual, ambas con dos pasadas, y el salto real ocurre en 512 KB.

En el external hashing el número de particiones casi no mueve el costo de I/O, y lo mueve en sentido contrario. El costo es fijo en unas $3P$ páginas: se lee la relación una vez, se escriben las particiones una vez y se vuelven a leer en la fase 2, 7 846 páginas con 15 particiones. Al subir k el I/O crece apenas, hasta 8 090 páginas con 255, y la razón es que cada partición desperdicia en promedio media página final. El riesgo verdadero está en el otro extremo: si k es demasiado chico, la tabla hash de alguna partición no cabe en RAM y hay que particionar de nuevo, y cada nivel extra cuesta otras $2P$ páginas.

Comparado con PostgreSQL, mi implementación es entre ocho y diecinueve veces más lenta: el motor ordenó en 115,7 ms y agrupó en 44,1 ms, mientras que Python necesitó 2,21 s y 0,68 s con el mismo presupuesto de memoria. La diferencia no está en el I/O, que es del mismo orden en ambos casos: PostgreSQL movió unos 112 MB de archivos temporales en el ORDER BY y mi

sort unos 135 MB. Está en la CPU y en el formato. El motor está escrito en C, compara fechas como enteros de cuatro bytes y mantiene las tuplas en su representación binaria, mientras que Python vuelve a interpretar cada tupla con struct y decodifica seis cadenas por registro en cada pasada. A eso se suma que PostgreSQL repartió el trabajo entre dos procesos y leyó las tablas desde el buffer pool ya caliente, sin I/O físico. Proyectar solo las columnas necesarias antes de ordenar o particionar, como hace el motor, sería la optimización más rentable de mi versión.

Entre los dos algoritmos, preferiría external sorting cuando el orden importa o se reutiliza más adelante: un ORDER BY, un DISTINCT que deba salir ordenado, la construcción de un índice B+ o un merge join posterior. También cuando hay sesgo en los datos, porque su costo no depende de cómo se distribuyan las claves, y cuando la cantidad de grupos distintos se acerca al número de tuplas, caso en el que la tabla hash dejaría de caber. External hashing conviene cuando basta la igualdad y los grupos son pocos frente a las tuplas, que es justo lo que pasa acá: 331 603 tuplas se resumen en 6393 grupos, el hashing cuesta unas 3P páginas contra 8P del sort y termina en menos de un tercio del tiempo. Su punto débil es la sensibilidad al sesgo, ya que una clave muy frecuente desbalancea una partición, y que no sirve para rangos, desigualdades ni para producir salida ordenada.

Conclusiones

Los tres planes de la primera parte muestran los mismos algoritmos que se implementaron en la segunda: external merge sort en el ORDER BY y en el Sort del GROUP BY, y external hashing tanto en el HashAggregate desbordado como en los 512 batches del Hash Join. La implementación en Python reproduce ese comportamiento con las mismas restricciones de memoria y, en el caso del GROUP BY, llega exactamente al mismo resultado que el motor. Lo que más se nota al variar BUFFER_SIZE es que la memoria solo ayuda al sort cuando alcanza para eliminar una pasada de merge, mientras que el hashing se mantiene plano porque su costo de I/O ya es el mínimo posible para el problema.

Anexo: archivos entregados

Junto con este informe van los tres módulos que pide el enunciado. `heap_file.py` implementa `export_to_heap`, `read_page`, `write_page` y `count_pages` sobre la clase `HeapFile`. `external_sort.py` implementa `generate_runs`, `multiway_merge` y `external_sort`, más la verificación de orden de la salida. `external_hashing.py` implementa `partition_data`, `aggregate_partitions` y `external_hash_group_by`, junto con las funciones `h_p` y `h_r` y la tabla hash con encadenamiento. Los datos van en `data/employee.bin` y `data/department_employee.bin`, exportados desde PostgreSQL con el formato paginado que describe la sección 2.1. Cada módulo se ejecuta por sí solo y vuelve a imprimir las salidas que aparecen en las capturas de este informe.